

Cryptography Coursework Report

Module: Applied Cryptography

Student Project: SecureVault

a) Title of Project

SecureVault: A Multi-Layer, From-Scratch Cryptographic Communication System Using Twofish, RC4, ElGamal, and Elliptic-Curve Diffie-Hellman

b) Introduction and Problem Statement

In this digital age where data is king, ensuring data integrity during transit and storage is not a mere amenity but a necessity for almost every industry known to man. Applied cryptography is the branch of science that provides us with the mathematical techniques needed to encrypt messages such that it is protected from unauthorized access and interception during transit, as well as tampering while in storage. It forms the basis of secure online transactions such as internet banking, confidential patient records according to HIPAA & GDPR, government communications that require authentication, as well as private corporate data (intellectual property).

The problem this project aims to solve are data confidentiality while in transit and data integrity while in rest. The problem where a sensitive message is being sent between two parties through an unreliable network (such as the internet) is prone to interception by a third party (adversary). If the message is not encrypted, it can be read entirely by the adversary while if the message is being encrypted but there is no data integrity check, the adversary might alter it, and we have no way to know. Eavesdropping and tampering thus form two very prominent threats that the SecureVault aims to counter.

The relevance of these issues can be seen in a variety of real world situations. In the health sector, a patient record being transmitted between a hospital and the insurance agency must be kept confidential for reasons of patient privacy and to prevent breaches in data protection laws. Similarly, inter-bank transfer must be protected by encryption and authentication to prevent fraudulent transactions. In the government, sensitive communication lines must be encrypted by verifiable algorithms in order to avoid espionage. In the IBM Cost of a Data Breach Report (2023), the average cost of a data breach recorded an increase of 15.1% and now stand at \$4.45 million US dollars.

This project builds the multi-layer secure communication pipeline called SecureVault where all cryptographic primitives are completely built from scratch, without any reliance on external cryptographic libraries (e.g. PyCryptodome, OpenSSL). This limitation helps each mathematical step to be visible, verifiable and explainable, including the multiplication within GF(2) on Twofish cipher and addition on elliptic curves at P-256.

c) Project Objectives

1. Write two symmetrical encryption algorithm from zero for the safe storage of data and for inner layer of encryption: Twofish, a 128 bit block cipher with an 8-round Feistel network, MDS matrix diffusion and PKCS7 padding; RC4, a stream cipher which has Key Scheduling Algorithm, Pseudo-Random Generation Algorithm and XOR keystream generation.
2. Write one asymmetrical algorithm for secure transmission of data: ElGamal encryption with a 256-bit prime field, use of Miller-Rabin primality test, search for the primitive root with Euler's criterion and use of Extended Euclidean modular inverse, to wrap-up the symmetrically encrypted message bundle to get it readable by only intended receiver.
3. Combine both, to create a multiparty system with certificate based authentication: exchanging of public keys between Alice and Bob, performing ECDH (secp256r1 / NIST P-256) key agreement, derivation of symmetric (Twofish and RC4) session keys with HKDF-MD5 and exchange of messages via layer encrypted pipeline with MD5 based integrity checks.
4. Carry out performance analysis (computation & communication cost): individually benchmarks the implemented encryption algorithm with `time.perf_counter` in Python, calculating computation throughput in bytes/sec and per operation latency; also reporting the communication cost in terms of ciphertext expansion factor.

d) Literature Review

Cryptography is a well-researched and documented field. The following ten sources were referenced to build upon the existing literature and help to frame the problem this project addresses.

1. Rivest, R. (1994). The RC4 Encryption Algorithm. In this specification document, RSA Security provides a detailed account of the KSA and PRGA elements of the RC4 algorithm. Though it should be noted that RC4 has weaknesses (especially when employed with certain modes like WEP), these are

addressed in this project as RC4 is only used as an external layer wrapping the already-encrypted Twofish ciphertext and never for raw plaintext encryption.

2. Schneier, B. Et al. (1998). Twofish: A 128-bit Block Cipher. This is the original paper submitting Twofish as an AES candidate where the design of the Feistel structure, the key dependent S-boxes, MDS matrix and the PHT is described. The design is followed closely for the implementation of SimplifiedTwofish in this project.

3. Elgamal, T. (1985). A Public Key Cryptosystem and a Signature Scheme Based on Discrete Logarithms. Elgamal's groundbreaking paper in which he introduces his public-key cryptosystem based on the Discrete Logarithm Problem (DLP).

4. Koblitz, N. (1987). Elliptic Curve Cryptosystems. One of the early papers introducing ECC, where it is shown that elliptic-curve groups are capable of supporting Diffie-Hellman key exchange with around 128-bit security at only 256-bit key lengths compared to RSA or other finite-field DH methods.

5. National Institute of Standards and Technology (NIST). (2013). FIPS 186-4: Digital Signature Standard - Appendix D. Details the standard parameters used for secp256r1 (P-256) including the prime P, the parameters A and B, the base point (Gx, Gy) and the order of the group N. These parameters are hardcoded into `ecdh_kdf.py`.

6. Krawczyk, H., Bellare, M., Canetti, R. (1997). HMAC: Keyed-Hashing for Message Authentication. This paper defines the HMAC construction which is constructed by hashing the message with a key which is repeated on both sides: $H((K \parallel \text{opad}) \parallel H((K \parallel \text{ipad}) \parallel m))$

and has been implemented for the derivation of the key. This will be the building block for the HKDF-MD5 key derivation.

7. Krawczyk, H., Eronen, P. (2010). HMAC-based Extract-and-Expand Key Derivation Function (HKDF). The paper that standardises the Extract and Expand model, used in the `kdf_derive()` function where MD5-HMAC is used to obtain the independent keys for the Twofish and RC4 components.

8. Rivest, R. L. (1992). The MD5 Message-Digest Algorithm. This RFC describes

the MD5 algorithm in detail, the 4 round, 64 step hash function along with all constants, intermediate variables and the compression function which have been reimplemented in python without the use of outside libraries.

9. Bernstein, D. J., Lange, T. (2017). Post-Quantum Cryptography. A survey of Post-Quantum Cryptography, that includes a brief discussion of why ECC is vulnerable to Shor's algorithm on powerful enough quantum computers which serves as motivation for the optional PQC component.

10. IBM Security. (2023). *Cost of a Data Breach Report 2023*. Industry report quantifying the financial and reputational cost of weak cryptographic implementations, providing real-world justification for the engineering effort of the SecureVault system.

Identified Gap

Most existing educational cryptography projects use high-level libraries (PyCryptodome, cryptography.io, etc.) that abstract every mathematical detail and make it impossible to check the correctness of each operation internally. SecureVault uses the low-level implementation of each algorithm-GF(2) arithmetic, point addition on elliptic curves, MD5 padding, HMAC, etc.-from first principles in around 1000 lines of python and depends on few standard python modules: math, struct and random. This makes it possible to check the algorithm operation step-by-step by hand.

e) Methodology

Implementation Method

All the above mentioned algorithms have been implemented using pure python3 with only standard library math utilities being imported. The project consists of 6 files which include:

- **rc4.py**: RC4 Stream Cipher (KSA+PRGA+XOR)
- **twofish.py**: Twofish Block Cipher (16 round Feistel+MDS+PHT)
- **elgamal.py**: ElGamal Asymmetric(Miller-Rabin, Primitive Root)
- **ecdh_kdf.py**: ECDH P-256 + HKDF (HMAC-MD5)
- **md5.py**: MD5 Hash (64 step compression)
- **diffie_hellman.py**: Classical DH (768 bit group)

Manual Verification

RC4 (Symmetric): Using a toy 4 element S-box we verify our logic for the swapping mechanism

as well as the generation of the keystream.

Elgamal (Asymmetric): Using small prime arithmetic ($p = 23$, $g = 5$) we verify the round trip of our encrypter and decrypter. In order to check our decryption, we check if it returns original value using modular inverse.

Twofish (Symmetric): Verified by calculating a single Pseudo-Hadamard Transform (PHT) by hand

Performance Measurement

We measure performance using time. PerfcounterNs(). The report includes following measures:

- throughput (MB/s)
- latency (ms)
- cipher text expansion ratio

Testing Strategy

Unit tests for each module are standalone. The whole pipeline is verified by simulate. Py that includes layered encryption, decryption identity checks, and active tamper attack simulation.

g) Use Case and Scenario Design

Business Case: SecureVault Corporate Communication Portal

The scenario is Alice sends a confidential money transfer order to Bob. This communication should be invisible to a third party (Eve) and should be tamper proof.

Communication Sequence

1. Identity: Bob publishes his ElGamal Public Key (verified by a simulated CA).
2. Agreement: Alice and Bob perform ECDH to generate a shared secret.
3. Derivation: HKDF-MD5 splits the secret into a Twofish Key and an RC4 Key.
4. Encryption: * Layer 1: Alice encrypts the message with Twofish (at-rest style).
 - a. Layer 2: Alice encrypts the result with RC4 (in-transit style).
5. Integrity: Alice attaches an MD5 hash of the original plaintext.
6. Transmission: Alice sends the bundle.
7. Decryption: Bob uses his ElGamal Private Key to verify Alice's handshake, then reverses the RC4 and Twofish layers to recover the message.

h) Ethical, Legal, and Professional Considerations

- **Data Integrity** - All simulation data is synthetic, no actual PII or sensitive corporate data is utilized.

- **Legal** - The architecture complies with GDPR and the UK Computer Misuse Act 1990.
- **Responsible Disclosure** - Acknowledges the weaknesses of RC4 and MD5 and states they are for instructional layering/integrity only.
- **Transparency** - No "security-through-obscurity". All math is shown and traceable.

i) Conclusion

The SecureVault project can be viewed as a demonstration that it is indeed possible to build a full, robust, multi-layer cryptocommunications system purely from math fundamentals. As each primitive in the system is hand coded the project has an undeniable knowledge of the mathematics it relies on on a daily basis. The executable end-to-end simulation serves as an undeniable proof that the system is indeed keeping secrets safe and data honest while making it possible for anyone to look inside.